

UC LINA 600 LINEAR ALGEBRA FOR GRAPHICS · MODULE 11 · LECTURE NOTES

Quaternions

How rotations are actually stored.

MODULE OUTCOMES

- Explain what is wrong with Euler angles.
- State the quaternion representation of a rotation.
- Compose and apply rotations with quaternions.
- Interpolate rotations correctly.
- Choose a rotation representation for a purpose.

1 What is wrong with angles

EULER ANGLES ARE ORDER-DEPENDENT, INTERPOLATE BADLY, AND LOSE A DEGREE OF FREEDOM AT CERTAIN ORIENTATIONS

Order-dependent: 'yaw 30 degrees then pitch 40' and 'pitch 40 then yaw 30' produce different orientations (Module 03 §2's non-commutativity) — **so the convention has to be stated, and there are a dozen conventions in common use**, which is a genuine interoperability problem.

Bad interpolation: interpolating the three angles separately does not produce a smooth rotation — the result wobbles, changes speed, and **takes a path that depends on the convention** rather than on the two endpoints.

And gimbal lock: at certain orientations two of the three rotation axes align, and a degree of freedom is lost — the system can no longer rotate in one direction without a discontinuous jump — **which is not a bug in anybody's implementation but a property of the representation itself**.

So Euler angles are perfectly good for a user interface and poor for storage or interpolation — which is exactly the division of labour that engines use (§4), and is why the answer is not to pick one representation.

2 The representation

$$\mathbf{q} = (\cos(\theta/2), \quad \sin(\theta/2) \cdot \mathbf{axis})$$

- A scalar part and a vector part, four numbers total. The axis is the eigenvector with eigenvalue 1 (Module 10 §1) — the direction the rotation leaves alone — **and the half-angle is the part that surprises everybody** and has to be remembered.

- **Composition is quaternion multiplication, and it is not commutative — which matches the fact that rotations do not commute** (Module 03 §2) — **so the algebra records the geometry faithfully**, as it did for matrices.
- **And q and $-q$ represent the same rotation** — the double cover — which is harmless for applying a rotation and **matters a great deal for interpolation: pick whichever of the two is nearer to your starting quaternion before interpolating, or you take the long way around the sphere.**
- **Half the angle, and q and $-q$ are the same rotation — two facts that account for most quaternion confusion, and both of which matter. Picking the nearer of q and $-q$ before interpolating is the bug everybody writes once**, and it shows up as a character spinning 350 degrees the wrong way between two keyframes.

3 Why engines use them

- **No gimbal lock** — every orientation is represented equally well, with no degenerate configurations anywhere on the sphere, which removes an entire class of bug.
- **Smooth interpolation** — spherical linear interpolation (slerp) follows the shortest arc between two orientations at constant angular speed, which is exactly what animation needs and what Euler interpolation fails to provide.
- **Cheap composition and cheap renormalisation** — four numbers rather than nine, and drift from accumulated floating-point error is corrected by a single normalise rather than by a Gram-Schmidt pass over a matrix (Module 05 §4).
- **And numerical stability under repeated composition** — which matters in an animation system accumulating thousands of operations per second per skeleton, where a matrix would drift measurably away from orthogonality.
- **At the cost of being essentially unreadable** — nobody can look at four numbers and picture the rotation they represent — which is the one real cost, and is why engines store quaternions internally and display Euler angles in the inspector.

4 Choosing

Representation	Use it for	Do not use it for
Euler angles	User interfaces, authoring tools, configuration files, anything a human types.	Storage, interpolation, or composition (§1).
Rotation matrix	Applying the rotation to many vectors, and uploading to the GPU, which wants a matrix.	Long-term storage — nine numbers, and it drifts out of orthogonality.
Quaternion	Storage, interpolation, and composition (§3).	Being read or edited by a human.

Convert at the boundaries — author in Euler angles, store and interpolate as a quaternion, convert to a matrix when uploading to the GPU — which is what every engine actually does, and is why all three conversions exist in every maths library. **The three-representation workflow is the practical answer, rather than choosing one and living with its weaknesses.**

AND THE HONEST SUMMARY

Quaternions are not mysterious: they are a half-angle axis-angle encoding together with a multiplication rule that happens to compose rotations correctly — and the half-angle is the only genuinely odd part of the construction.

You do not need the algebra's full derivation to use them — the relationship to complex numbers and to $SU(2)$ is interesting and is not required — **you need the construction, the composition rule, slerp, and the double-cover caveat.**

Which is four facts, and Project 2 accepts either a rotation matrix or a quaternion for the orbiting camera, provided you say which you used and why — the reasoning being the assessed part.

And the axis in §2 is Module 10's eigenvector — which is a satisfying place for the two modules to meet, and is worth verifying numerically once: build a rotation matrix, extract its eigenvector with eigenvalue 1, and compare it to the quaternion's vector part.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
3Blue1Brown and Ben Eater — Visualizing quaternions (free, interactive)	§2 — the best explanation available, and it is interactive, which this topic needs more than most.
Shoemake — Animating rotation with quaternion curves	§3's slerp , in the original paper that introduced it to graphics.
Lengyel, chapter 4 (library copy)	§§2 to 4 — quaternions in engine conventions, with the conversions worked.
Hanson — Visualizing Quaternions	§2 in depth — far more than this course needs, and the right book if you want the full picture. Library copy.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Construct a gimbal lock configuration** and demonstrate the lost degree of freedom.
- 2 **Interpolate two orientations with Euler angles** and observe the wobble.
- 3 **Build a quaternion** from an axis and angle.
- 4 **Verify the half-angle** by rotating a vector with it.
- 5 **Compose two quaternions** and compare to composing the equivalent matrices.

- 6 **Confirm q and $-q$ rotate identically.**
- 7 **Implement `slerp`**, and then omit the nearer-of-two check and watch it take the long way.
- 8 **Convert quaternion to matrix and back**, and check the round trip.
- 9 **Compose a thousand rotations** as matrices and as quaternions, and compare the drift.
- 10 **Extract a rotation matrix's eigenvector** and compare it to the quaternion axis.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the three problems with Euler angles.
- 2 What is gimbal lock, and is it an implementation bug?
- 3 Give the quaternion construction from axis and angle.
- 4 What is the axis, in Module 10's terms?
- 5 Why is composition non-commutative, and is that right?
- 6 What is the double cover, and when does it matter?
- 7 Name four advantages of quaternions.
- 8 What is their one real cost?
- 9 Give the three representations and the job of each.
- 10 What four facts do you actually need to use them?